⛰️ HikerNet Trekking Module: In-Depth Report

The Trekking Module is the backbone of HikerNet, designed to facilitate a robust, safe, and engaging environment for outdoor enthusiasts. It aggregates capabilities for real-time tracking, geospatial data management, social planning (group treks), safety monitoring, and competitive gamification (leaderboards).

🕒 The Past: Evolution and Iteration

Based on the Git commit history from January 6 to February 12, 2026, the module evolved dramatically from a standard CRUD concept to a sophisticated geospatial application.

1. Week 3 (Jan 15–19): Core Foundations The trek module started with the basic foundation structure. Data models were initially established, adopting standard coordinates. Soon thereafter, **GeoJSON** arrays and geospatial indices were introduced to map continuous trails. Basic trek tracking and statistical calculations like distance and duration were integrated into the backend and exposed to early React Native UI screens.

2. Week 4 (Jan 19–26): The Group Synergy & Mapping Update A major leap occurred when the team successfully augmented the solo track feature with **Group Trek Rooms**. This required the implementation of the `Room` system (with unique 7-digit access codes), invitation loops, and member-based status validations (e.g., Ready/Not Ready flow). Concurrently, **Trail Visualizations** were improved using MapLibre for React Native and Leaflet on the web, enabling satellite views, custom icon pins for waypoints, and clearer path rendering.

3. Week 5–6 (Jan 26 - Feb 2): Competitive & Safety Integration The backend was tied with live Leaderboards tracking total distance. This gamified the module heavily. Additionally, this period saw the integration of location-based Weather integrations (warning users dynamically about conditions on the trail).

4. Week 8 (Feb 7–12): Trek Logic Overhauls & automated POIs Early February saw strict refinements applied to the Trek GPS tracking algorithm for better accuracy, as well as more granular elevation gain logic. This was combined with the novel Trail Discovery system, connecting the app directly to OpenStreetMap (OSM) via `osmtogeojson` to pull local trails dynamically.

⚡ The Present: Current Architecture

At present, the Trekking Module consists of seamlessly integrated backend computations and frontend sensor listening layers, utilizing cross-platform compatibility.

⚙️ Backend Logic & Infrastructure

Data Schema (`Backend/src/models/Trek.js`)

- **Geospatial Processing**: Built completely using Mongo's 2dsphere indexing and GeoJSON arrays. The trail format is saved as a `LineString` consisting of `[[longitude, latitude]]`. The schema index `trekSchema.index({ path: '2dsphere' });` allows the system to easily query `$near` and `$geoWithin` conditions, driving trail discovery routines.
- **Embedded Waypoints**: The system tracks dynamic user actions during the trek. Waypoints are embedded arrays on the model holding precise `{latitude, longitude, altitude}`, string descriptions, customizable icon parameters, and arrays of photos uploaded directly from the trail (integrated with Cloudinary via `cloudinary.js`).
- **Stats Calculation Engine**: Metrics track continuously via real-time computations:
  - `distance` (sum of point-to-point line segments in meters).

    - `duration` (computed in seconds from start).
    - `elevationGain` (calculated using deltas derived from device barometer and GPS altitude).
    - `avgSpeed` (calculated via duration and distance on-the-fly).
- **Group Architecture**: Driven by `Room.js`, a user becomes the `leader` of a session, and issues `roomCode` tokens. It acts as an orchestrator tying multiple `User` object IDs to a single `Trek` session while retaining member acceptance layers before starting up the recording algorithm.

APIs & Controllers (`/api/treks`, `/api/rooms`)

- `POST /start` & `PATCH /:id/end`: Controls state progression (ongoing → paused → completed).
- `POST /:id/waypoint` & `POST /:id/images`: Handles mid-hike state augmentation.
- Automated **Cron Jobs** exist to clear stale data, refresh OSM tracking arrays, and parse external trails.

📱 Frontend Implementation & UI

Sensory and Location Data (`Frontend/app/trek/`)

- The React Native layer hooks into device-level APIs natively via `expo-location` and `expo-sensors`. It acts as a constant emission client while tracking is active. In `active.jsx`, coordinate arrays pool in React State momentarily before executing chunked HTTP payloads to `/api/treks/:id/waypoint`.
- **Render Engine**: Using MapLibre Native and React Leaflet (for web environments), the UI accurately reflects line paths over interactive MapViews. It relies heavily on Worklets via React Native Reanimated to prevent tracking states from blocking UI (maintaining a silky 60fps rendering layer).

Trekking UX Flow

- **Live Trek Card**: During an ongoing session, this card dominates the `explore.jsx` view. It persistently updates speed, altitude, and distance.
- **Group Lobby (`room-join.jsx` & `lobby.jsx`)**: When entering a group trek, users sit in a lobby where sockets/polling update ready states of participants. Minimal trek starts are forced off UI toggles tied strictly to the group leader's device API calls.
- **Post-Trek Review (`[id].jsx`)**: Upon completion, a history screen drops down showing a highly polished visual display of the GeoJSON trail wrapped over the static map with a complete widget covering the final stats.

🐛 Known Problems, Errors & Accuracy Issues

Despite the module's success, several bugs and logic inaccuracies were logged and partially addressed during active development. Ongoing attention is required in these areas:

1. GPS Tracking Accuracy & Signal Drop

- **The Issue**: Early iterations recorded severe tracking inaccuracies (`b30f352`), partly due to hardware inconsistencies drawing rapid irregular polygons rather than straight lines.
- **Current Mitigation**: A "Smart Location" custom hook (`useSmartLocation.js`) now utilizes sensor fusion. It applies an accuracy filter (rejecting coordinates with raw accuracy > 100 meters) to account for canopy cover/indoor signal blocking.
- **Stationary Drift**: Pedometer gating has been introduced to reject small GPS drifts (< 0.5 meters) when the user is not actively taking steps, preventing false distance additions.

2. Group Trek edge cases

- **General Desynchronization**: Functionality errors occurred around participants dropping from active sessions or photo/waypoint deletions replicating inconsistently (`8725dff`). These features required reversing and recalibration.
- **API Polling Overhead**: Currently, the lobby falls back to API polling if sockets fail, which generates significant overhead payload.

3. General Backend Bottlenecks

- Occasional errors regarding payload sizes when attempting to upload high-fidelity image arrays concurrently from dense trail waypoints.
- Early route logic produced errors during user disconnections which necessitated minor API restructuring (`a05dc49`).

🔮 The Future: Roadmap & Evolutions

Reviewing the App Analysis and the development work plans mapping, here's how the module is strictly outlined to evolve going forward:

1. Immediate Sprint Actions (Short-term)

- **Offline Reliability:** Right now, connections need to remain partly active to save state efficiently. The immediate future dictates pushing caching capabilities via `expo-file-system` and async storage to cache static tiles natively, allowing users to safely log miles deep within forests without cell tower connections, subsequently syncing paths securely on WiFi reconnects.
- **Push Notification Live Links:** Incorporating push-based deep links so that group invites immediately pull members into active lobby sessions without refreshing.

2. 2-Month Evolution (Medium-term)

- **AI-Powered Recommendation Engines:** Applying ML algorithms natively against the Mongo 2dsphere indexes. Based on past recorded behavior (e.g., speed, and preference for steep elevation), the application will inject automated dynamic trail cards utilizing OSM data into the `discover` feed.
- **Social Gamification:** Creating dynamic Achievement Badges computed against the statistics engine (e.g., "10,000 ft ascent", "100 km tracked") and community competitive challenges.

3. Structural Revisions (Long-term)

- **WebSocket Streaming:** Currently relies on efficient API intervals. Eventually, group sessions will run entirely on standard WebSocket connections for 1-to-1 live peer dot tracking over the native map (creating real-time visual collaboration on the trail).
- **Physical Wearable Layers:** Porting `active.jsx` constraints out via Bluetooth Low Energy connections, creating hooks to push/pull biometric states to smartwatches (Apple Watch/WearOS) to augment standard location tracking.
- **Marketplace Expansion:** Using the finalized and stable trekking algorithm behavior as a foundational bedrock to implement guided trek bookings directly inside the map ecosystem.